

Image Classification

Image Classification

1. Model Introduction
2. Image Classification: Image
Effect Preview
3. Image Classification: Video
Effect Preview
5. Image Classification: Real-time Detection (CSI Camera)
Effect Preview
4. Image Classification: Real-time Detection (USB Camera)
Effect Preview

Use Python to demonstrate **Ultralytics**: Image classification effects on images, videos, and real-time detection.

1. Model Introduction

Image classification is the simplest of the three tasks, involving classifying an entire image into one of a set of predefined categories.

The output of an image classifier is a single class label and confidence score. Image classification is very useful when you only need to know which category an image belongs to, without needing to know the location or exact shape of objects in that category.

2. Image Classification: Image

Use yolo26n-cls_ncnn_model to predict images in the ultralytics26 folder (not images that come with ultralytics).

Enter the code folder:

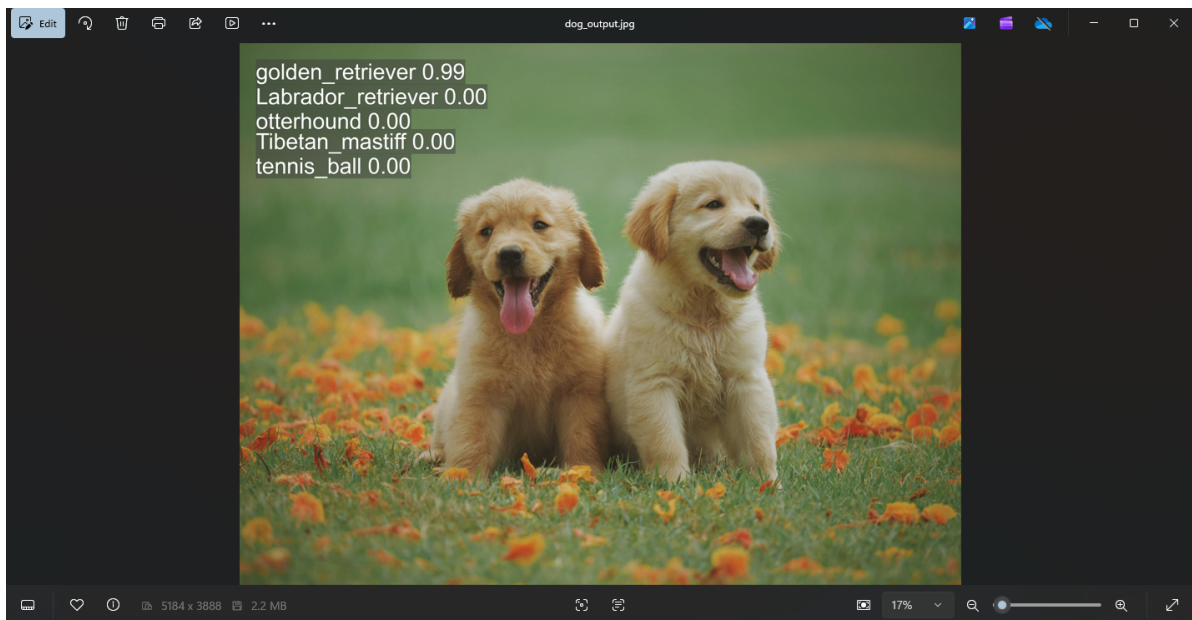
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 04.classification_image.py
```

Effect Preview

yolo recognition output image location: /root/yolo26_data/output/



Sample code:

```
from ultralytics import YOLO

# Load a model
#model = YOLO("/ultralytics/yolo26n-cls.pt")
model = YOLO("/ultralytics/yolo26n-cls_ncnn_model")

# Run batched inference on a list of images
results = model("/ultralytics/assets/dog.jpg") # return a list of Results objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/root/yolo26_data/output/dog_output.jpg") # save to disk
```

3. Image Classification: Video

Use yolo26n-cls_ncnn_model to predict videos in the ultralytics26 folder (not videos that come with ultralytics).

Enter the code folder:

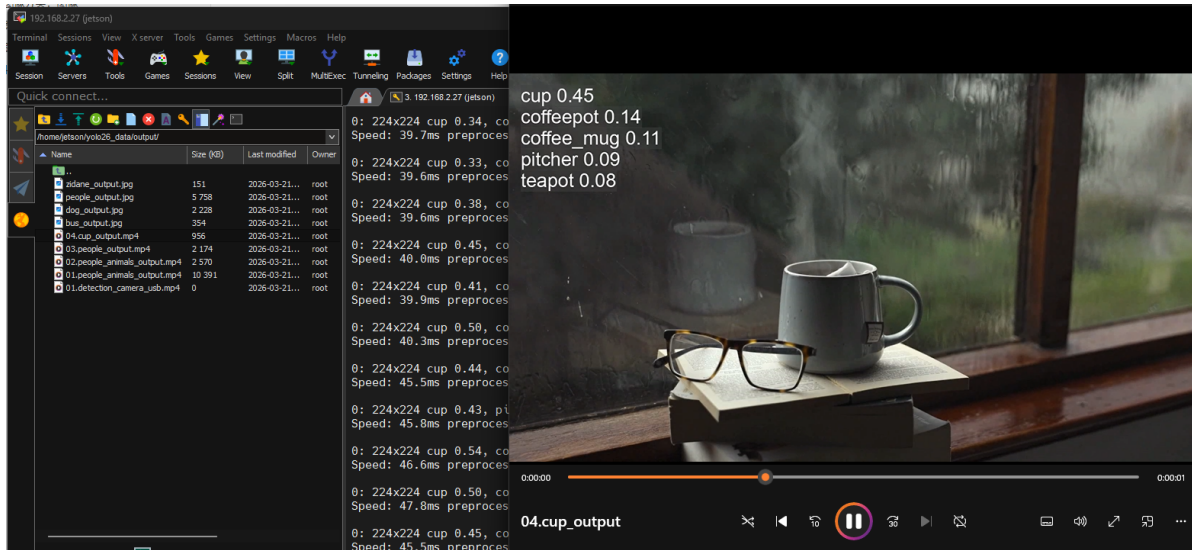
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 04.classification_video.py
```

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
#model = YOLO("/ultralytics/yolo26n-cls.pt")
model = YOLO("/root/yolo26_data/models/yolo26n-cls.pt")

# Open the video file
video_path = "/root/yolo26_data/cup_s.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/root/yolo26_data/output/04.cup_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)
```

```

# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

5. Image Classification: Real-time Detection (CSI Camera)

Use yolo26n-cls for real-time image classification through CSI camera.

Note: Please ensure CSI camera is properly connected and use docker startup script that supports CSI camera to enter container

Enter the code folder:

```
cd /ultraalytics/yahboom_demo
```

Run the code:

```
python3 04.classification_camera_csi.py
```

Press keyboard **q** key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```

import cv2
from ultraalytics import YOLO

def gstreamer_pipeline(sensor_id=0, width=640, height=480, fps=30):
    return (
        f"nvarguscamerasrc sensor-id={sensor_id} ! "
        f"video/x-raw(memory=NVMM),width={width},height={height},framerate=
{fps}/1 ! "
        "nvvidconv ! "
        "video/x-raw,format=BGRx ! "
        "videoconvert ! "
        "video/x-raw,format=BGR ! appsink"
    )

```

```

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-cls.pt")

# Open the camera
frame_width = 640
frame_height = 480
fps = 30
cap = cv2.VideoCapture(
    gstreamer_pipeline(width=frame_width, height=frame_height, fps=fps),
    cv2.CAP_GSTREAMER,
)

if not cap.isOpened():
    raise RuntimeError("Failed to open CSI camera")

# Define the codec and create a Videowriter object to output the processed video
output_path = "/root/yolo26_data/output/04.classification_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the camera frame cannot be read
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Image Classification: Real-time Detection (USB Camera)

Use yolo26n-cls for real-time image classification through USB camera.

Note: Please ensure USB camera is properly connected to JetsonNano

Enter the code folder:

```
cd /ultralitics/yahboom_demo
```

Run the code:

```
python3 04.classification_camera_usb.py
```

Press keyboard **q** key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-cls.pt")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/root/yolo26_data/output/04.classification_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)
```

```
# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```